Table des matières

Labo 05 OLTP — CTE et récursivité sur le transactionnel 1
Le terrain, en une minute 1
Partie 1 — De la sous-requête à la CTE 2
Partie 2 — Chaîner, générer, parcourir 3
Partie 3 — Ce que le contraste vous a montré 6
Et ensuite ? 7

Labo 05 OLTP — CTE et récursivité sur le transactionnel

Laboratoire du Cours 05, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On refait le laboratoire OLAP, question pour question, mais sur l'autre terrain : la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Comptez trois quarts d'heure.

Les deux voies — Ce labo est le jumeau de `Labo_05_OLAP_SQL.md`. Les questions sont les mêmes ; le schéma, lui, ne l'est pas. Gardez les deux fichiers ouverts côte à côte : c'est en comparant deux requêtes qui répondent à la même chose que le contraste devient visible.

Identifiants en minuscules — Le portage PostgreSQL d'AdventureWorks range les tables dans des schémas en minuscules (`production`, `sales`) et replie les noms de colonnes de la même façon. Si une requête copiée d'ailleurs échoue sur un nom de colonne, c'est presque toujours ça.

Le terrain, en une minute

Deux tables du Labo 04 reviennent, et une nouvelle arrive pour la récursivité :

Table	Ce qu'elle contient	Grain
`sales.salesorderheader`	l'en-tête d'une commande : client, dates, totaux	une commande
`sales.salesorderdetail`	les lignes d'une commande : produit, quantité, prix	une ligne de commande
`production.product`	le catalogue : nom, couleur, prix de liste	un produit

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Grain
`production.billofmaterials`	la nomenclature : quel composant entre dans quel assemblage	un lien assemblage vers composant

`production.billofmaterials` est la table qui compte aujourd'hui. Elle se réfère à elle-même : `productassemblyid` désigne l'assemblage, `componentid` la pièce qui y entre, et les deux pointent vers `production.product`. Un vélo contient un cadre, qui contient un tube, qui contient un matériau. C'est un arbre, et un arbre se parcourt par récursion.

```
SELECT COUNT(*) AS lignes_bom,
       COUNT(*) FILTER (WHERE enddate IS NULL) AS lignes_actives
FROM production.billofmaterials;                -- 2 679 lignes, dont 2 480 actives

SELECT COUNT(*) FROM production.billofmaterials
WHERE productassemblyid IS NULL;                -- 103 produits finis, les racines de l'arbre
```

Une ligne dont `enddate` n'est pas nul décrit une nomenclature périmée. On filtre donc systématiquement sur `enddate IS NULL`, sans quoi la récursion ramène d'anciennes versions du produit mêlées aux actuelles.

Partie 1 — De la sous-requête à la CTE

Théorie : `Theorie_05_SQL.md`, sections 1 et 2.

Même exercice que sur l'entrepôt, sur les ventes mensuelles. Version imbriquée :

```
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois) AS ventes_mois_prec
FROM (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
) AS mensuel
ORDER BY annee, mois;
```

Version CTE :

```
WITH mensuel AS (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
```


© 2026 iAlexMG. Tous droits réservés.

```
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
)
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois) AS ventes_mois_prec
FROM mensuel
ORDER BY annee, mois;
```

Observez. Le gain de lisibilité est le même que sur l'entrepôt, et il est même plus net ici : la sous-requête OLTP était plus longue, donc plus pénible à enjamber du regard. La CTE ne change rien au plan d'exécution, elle change ce que votre œil a à faire.

Partie 2 — Chaîner, générer, parcourir

2.1 — Chaîner deux CTE

Théorie : `Theorie_05_SQL.md`, section 3.

Objectif : les mois dont les ventes dépassent la moyenne mensuelle de l'entreprise.

```
WITH mensuel AS (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
),
moyenne AS (
    SELECT AVG(ventes) AS ventes_moyens FROM mensuel
)
SELECT m.annee, m.mois, m.ventes
FROM mensuel AS m
CROSS JOIN moyenne
WHERE m.ventes > moyenne.ventes_moyens
ORDER BY m.ventes DESC;
```

Observez. La requête est identique à celle de l'entrepôt à partir de la deuxième CTE. Seule l'étape qui va chercher les données a changé. C'est la forme habituelle du contraste : le raisonnement est portable, la couche d'accès ne l'est pas.

2.2 — Générer une série, et combler les trous

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_SQL.md`, sections 5 et 6.

La série pure. Le mot-clé `RECURSIVE` n'est pas optionnel ici : sans lui, PostgreSQL refuse la requête en disant que la relation `nombres` n'existe pas.

```sql
WITH RECURSIVE nombres AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nombres WHERE n < 12
)
SELECT n FROM nombres;
```

Maintenant l'usage concret. La base commence en cours d'année : les premiers mois de la première année n'ont aucune commande. Repérez d'abord la plage réelle de votre portage.

```sql
SELECT MIN(orderdate)::date AS premiere, MAX(orderdate)::date AS derniere
FROM sales.salesorderheader;      -- sur le portage courant : 2022-05-30 et 2025-06-29
```

Remplacez `2022` par la première année que vous venez d'obtenir. Le rapport brut saute les mois vides :

```sql
SELECT EXTRACT(MONTH FROM h.orderdate)::int AS mois, SUM(d.linetotal) AS ventes
FROM sales.salesorderheader AS h
JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
WHERE EXTRACT(YEAR FROM h.orderdate) = 2022
GROUP BY 1
ORDER BY 1;
```

On génère les douze mois, puis on raccroche :

```sql
WITH RECURSIVE mois AS (
    SELECT 1 AS m
    UNION ALL
    SELECT m + 1 FROM mois WHERE m < 12
),
ventes_annee AS (
    SELECT EXTRACT(MONTH FROM h.orderdate)::int AS mois, SUM(d.linetotal) AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    WHERE EXTRACT(YEAR FROM h.orderdate) = 2022
    GROUP BY 1
)
SELECT mois.m AS mois, COALESCE(v.ventes, 0) AS ventes
FROM mois
LEFT JOIN ventes_annee AS v ON v.mois = mois.m
ORDER BY mois.m;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Les mois antérieurs à la première commande sortent à zéro, exactement comme sur l'entrepôt. Le mécanisme est le même : on part d'un axe complet, puis on y accroche des mesures incomplètes.

PostgreSQL offre un raccourci, et l'entrepôt a exactement le même. La CTE récursive `mois` se remplace des deux côtés par une seule ligne :

```
WITH mois AS (
    SELECT generate_series(1, 12) AS m
)
SELECT * FROM mois;
```

Retenez les deux écritures. `generate_series` est plus court et plus rapide à taper au quotidien ; la CTE récursive marche partout, y compris là où l'on vous interdira d'ajouter une fonction. En entretien, c'est la seconde qu'on vous demandera d'écrire.

2.3 — Parcourir une hiérarchie (la nomenclature)

Théorie : `Theorie_05_SQL.md`, sections 5, 7 et 8.

Objectif : dérouler tous les composants d'un vélo, niveau par niveau.

L'entrepôt vous a fait remonter un organigramme grâce à `DimEmployee.ParentEmployeeKey`, une colonne qui pointe vers la même table. La base transactionnelle n'expose pas la hiérarchie des employés de cette façon : AdventureWorks y range la chaîne de commandement dans un chemin de type `hierarchyid`, que le portage PostgreSQL rend sous forme de texte. En revanche, elle offre un arbre autrement plus intéressant, la nomenclature de fabrication.

On part du produit 999, `Road-750 Black, 52`, un vélo complet.

```
WITH RECURSIVE nomenclature AS (
    -- ANCRE : les composants directs du velo
    SELECT b.componentid, b.perassemblyqty, 1 AS niveau
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999
      AND b.enddate IS NULL
    UNION ALL
    -- RECURSION : les composants des composants deja trouves
    SELECT b.componentid, b.perassemblyqty, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
)
SELECT n.niveau, p.name AS composant, n.perassemblyqty AS qte
FROM nomenclature AS n
JOIN production.product AS p ON p.productid = n.componentid
ORDER BY n.niveau, composant;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Vous obtenez quatre niveaux et 89 lignes. Le niveau 1 donne les grands ensembles (le cadre, les roues, le guidon), et chaque niveau suivant les décompose. C'est la même mécanique que l'organigramme, sur un tout autre objet : une ancre, `UNION ALL`, puis la jointure de la table sur la CTE.

Comptez les composants par niveau pour voir la forme de l'arbre :

```sql
WITH RECURSIVE nomenclature AS (
    SELECT b.componentid, 1 AS niveau
    FROM production.billofmaterials AS b
    WHERE b.productassemblyid = 999 AND b.enddate IS NULL
    UNION ALL
    SELECT b.componentid, n.niveau + 1
    FROM production.billofmaterials AS b
    JOIN nomenclature AS n ON b.productassemblyid = n.componentid
    WHERE b.enddate IS NULL
)
SELECT niveau, COUNT(*) AS nb_composants
FROM nomenclature
GROUP BY niveau
ORDER BY niveau;
```

Observez. 14, 50, 21 et 4 composants aux niveaux 1 à 4, soit les 89 lignes de la requête précédente : l'arbre s'élargit au deuxième niveau, puis s'amincit.

Enlevez maintenant `AND b.enddate IS NULL` des deux morceaux, et relancez. Le compte gonfle : 16, 64, 23 et 4, soit 107 lignes au lieu de 89. Vous venez de faire entrer dans l'arbre des nomenclatures périmées, celles des versions antérieures du produit, et les 18 lignes de trop se répartissent sur trois niveaux. Sur une base transactionnelle, une table d'historique porte presque toujours ce genre de colonne de validité, et l'oublier dans une récursion se paie cher, parce que l'erreur se propage à tous les niveaux suivants.

Piège — Une CTE récursive peut boucler pour de bon, ici comme sur l'entrepôt. Si une donnée corrompue faisait qu'un assemblage se contienne lui-même, la requête tournerait jusqu'à épuiser la mémoire du serveur : ni PostgreSQL ni DuckDB n'imposent de limite de profondeur, et l'essai a été fait, une récursion sans condition d'arrêt tourne encore trois minutes plus tard sans un mot. La parade se code, et elle se code des deux côtés : ajoutez `AND n.niveau < 10` dans la partie récursive, ou traînez un chemin de clés et refusez d'y rentrer deux fois. Certaines plateformes coupent seules au bout d'une centaine de tours, dont Snowflake d'après sa documentation ; ne comptez pas là-dessus.

Partie 3 — Ce que le contraste vous a montré

Théorie : `Theorie_05_SQL.md`, section 9.

Ce qui a changé	Voie OLAP (entrepôt)	Voie OLTP (transactionnel)
L'arbre à parcourir	`DimEmployee`, une colonne parent posée à plat par l'alimentation	`billofmaterials`, l'arbre tel que l'application le tient

© 2026 iAlexMG. Tous droits réservés.

Ce qui a changé	Voie OLAP (entrepôt)	Voie OLTP (transactionnel)
La validité des lignes	historisée dans la dimension, à lire à une date	à filtrer soi-même sur le lien (`enddate IS NULL`)
Le coût d'une récursion profonde	élevé : 100 000 tours y prennent 9,5 s	faible : les mêmes 100 000 tours prennent 42 ms
Ce qui n'a pas changé	ancre, `UNION ALL`, partie récursive, chaînage des CTE	idem, au caractère près
Le mot-clé `RECURSIVE`	obligatoire	obligatoire
Générer une série	`generate_series(1, 12)`, ou la CTE récursive	la même chose, au caractère près
La boucle sans fin	à borner soi-même, le moteur ne coupe pas	à borner soi-même, le moteur ne coupe pas

Les trois dernières lignes méritent qu'on s'y arrête, parce qu'elles ne disent rien : les deux colonnes y sont identiques. C'est le résultat du cours et il n'était pas acquis. Sur les CTE, il n'y a presque pas de dialecte à apprendre, et tout ce qui vous ralentira en passant d'un terrain à l'autre vient du modèle ou du moteur, jamais de la grammaire.

Une leçon de fond se cache dans les deux premières lignes du tableau. L'entrepôt vous a servi une hiérarchie déjà mise à plat : une colonne parent, un lien franc, aucune date sur le lien lui-même. Quelqu'un avait fait ce travail pour vous pendant l'alimentation, et c'est pourquoi la récursion y est plus courte à écrire. La base transactionnelle vous donne la matière brute, l'arbre tel que l'application le tient, avec ses dates de validité sur chaque lien.

Mais attention à la conclusion facile, et le labo jumeau vous en a averti : l'entrepôt n'a pas fait disparaître l'historique, il l'a déplacé. Les dates ont quitté le lien pour entrer dans la dimension elle-même, sous la forme d'une nouvelle ligne à chaque changement. Le piège y est même plus silencieux qu'ici, puisque rien dans la requête ne rappelle qu'il existe. Ce n'est donc pas que l'entrepôt soit plus simple, c'est que la difficulté y a changé de place, et qu'elle a été payée en amont par l'ingénieur qui a écrit l'alimentation. Ce métier, c'est le vôtre si vous allez vers l'analytics engineering.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_05_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vous savez maintenant dérouler un arbre des deux côtés, et vous avez vu ce que l'alimentation d'un entrepôt fait disparaître de difficultés. Le Cours 06 attaque le SQL du reporting : sous-totaux, totaux généraux et tableaux croisés, avec `GROUPING SETS`, `ROLLUP`, `CUBE` et `PIVOT`.

© 2026 iAlexMG. Tous droits réservés.